

1

A Deterministic Numerical Stack

2

N. E. Davis ~lagrev-nocfep
ZeroAttest, Groundwire Foundation

3

4

Abstract

5

6

Contents

7

1	Introduction	2
----------	---------------------	----------

8

2	Background and Literature	3
----------	----------------------------------	----------

9

2.1	BLAS as a pseudo-standard	4
-----	-------------------------------------	---

10

2.2	Sources of floating-point nondeterminism . . .	4
-----	--	---

11

2.3	Reproducibility	5
-----	---------------------------	---

12

3	Deterministic Architecture	6
----------	-----------------------------------	----------

13

3.1	The numerical stack	7
-----	-------------------------------	---

14

3.2	SoftBLAS and SoftUnum	10
-----	---------------------------------	----

15

3.2.1	SoftBLAS	10
-------	--------------------	----

16

3.2.2	SoftUnum	12
-------	--------------------	----

17

4	Types	12
----------	--------------	-----------

18

4.1	Verification	13
-----	------------------------	----

19

5	Contributions	14
----------	----------------------	-----------

20

6	Benchmarks	15
----------	-------------------	-----------

21

6.1	Transcendental accuracy	19
-----	-----------------------------------	----

22

6.2	The price of determinism	19
-----	------------------------------------	----

23	7 Future Work	19
24	8 Conclusion	20

1 Introduction

Urbit is a deterministic functional operating system whose state is a pure function of its event log (`~sorreg-namtyv` et al., 2016). An event log replay must recover the state exactly, on any conforming runtime, indefinitely. This is not a performance nicety; it is the property that makes the system auditable, upgradeable, and peer-verifiable. Determinism is a load-bearing property all the way down.

Numerical computing is one of the most slippery places for determinism to leak through its abstractions. Standard IEEE 754 floating-point arithmetic is not associative, as an example, but the situation is much more dire. The IEEE 754 standard (, 2019) specifies the result of each individual operation under a given rounding mode, but a real numerical program is a composition of operations, and the standard says little about composition. A fused multiply-add computes $a \cdot b + c$ with a single rounding where two separate operations would round twice; a vectorized dot product sums partial results in an order chosen by the hardware; and a threaded BLAS reassociates a reduction differently depending on how many cores happened to be available.¹ Each of these is standards-conformant while none is deterministic across hosts.

The treatment of floating-point representation in Hoon by `~lagrev-nocfep` (2024) established the foundation for Urbit’s deterministic numerical computing stack. This article expounds both the state of the art of Urbit’s basic numerical packaging and the next layer: dense linear algebra, where the nondeterminism of conventional BLAS and IEEE 754 floating-point mathematics are acute and consequential. We introduce SoftBLAS,²

¹Urbit computing is single-threaded which removes one possible source of nondeterminism.

²`urbit/SoftBLAS`

a software-defined BLAS package; SoftUnum,³ a software-defined universal number (unum) package; and other components of a C-based numerical stack that is deterministic and compliant with the Hoon specification. We further describe the “ray” data structure which facilitates the type dispatch for different numeric representations while preserving identity between Nock-specified atoms and C-recognized arrays.

The central conceit is architectural. In conventional high-performance computing, C (or Fortran) takes primacy: the C implementation is what runs, and correctness is in some sense whatever the C does.⁴ Urbit inverts this relationship. The Hoon/Nock ISA implementation is the specification; the C jet is an optimization that is *obligated* to produce the specification’s exact output for every input, and should be discarded silently whenever it cannot. Determinism then is not merely a testable, desirable property, but a mandate the Nock-side architecture cannot express the violation of.

2 Background and Literature

Although the solution of partial differential equations is one of the oldest significant applications of computing,⁵ no standard library emerged for decades—unsurprising given the churn of hardware architectures, the diversity of numerical methods, the lack of a common interface definition, the rapid evolution of programming languages, and the active development of linear algebraic algorithms.⁶ Given that situation, rigorous bit-wise determinism was a concern of vanishingly small importance, whether cross-platform or even for serial runs on the same machine. However, over time the advent of high-performance computing, the emergence of BLAS as a pseudo-standard, and the increasing importance of reproducibility in numerical computing have made determinism both more pressing and more achievable.

³sigilante/SoftUnum

⁴Verification and validation apply, of course.

⁵Charney, Fjortoft, and Neumann (1950).

⁶Even the size of a byte took decades to standardize.

2.1 blas as a pseudo-standard

The Basic Linear Algebra Subprograms originate with Lawson, Hanson, Kincaid, and Krogh's 1979 specification of vector operations on strided arrays (Lawson et al., 1979), later extended to matrix-vector (Level 2) and matrix-matrix (Level 3) routines. The levels track both chronology and (naïve) asymptotic cost: Level 1 is $O(n)$, Level 2 is $O(n^2)$, Level 3 is $O(n^3)$. BLAS is less a standard than a pseudo-standard—a widely honored set of names and calling conventions (`?axpy`, `?gemv`, `?gemm`, with type prefixes `s`, `d`, `c`, `z`) with many independent implementations. (The original BLAS predated floating-point standardization as IEEE 754 by several years.) That plurality is exactly the problem for a consensus system: two conforming BLAS libraries routinely disagree in the low bits, and the same library disagrees with itself across machines and thread counts. Despite this fuzziness, BLAS quickly took its place as the bedrock layer of numerical and scientific computation.

2.2 Sources of floating-point nondeterminism

Cross-host divergence in a BLAS computation has a small number of recurring causes:

- *Reduction order.* Floating-point addition is not associative, so a dot product or a `gemm` inner loop summed in a different order yields a different rounded result; tiled and multithreaded implementations choose that order at run time.
- *Fused multiply-add.* Hardware that contracts $a \cdot b + c$ to one rounding produces different bits than two roundings, and whether contraction happens is a compiler- and target-dependent decision.
- *Vectorization and instruction selection.* SIMD horizontal sums, x87 80-bit intermediates, and differing `libm`-style kernels for fast paths all perturb the last bits.
- *Transcendental functions.* The elementary functions are

not specified to the bit by IEEE 754 at all, and every platform `libm` differs in its implementation.

`~lagrev-nocfep` (2024) further enumerated other reasons that contemporary floating-point mathematics cannot be considered deterministic even when single-threaded on a particular hardware target and instruction set architecture.

2.3 Reproducibility

Numerical determinism, or for our purposes synonymously reproducibility, requires that a computation yield the same result to the bit regardless of the platform on which it is carried out. This would not necessarily require the same algorithmic implementation in all cases (since all correct algorithms should converge on the true value given enough runway), but in practice floating-point quirks force this consideration in almost all cases.

Projects like ReproBLAS have yielded reproducible summation (independent of summation order) through error-free transformations and pre-rounding (Demmel and Nguyen, 2013). Intel’s Conditional Numerical Reproducibility offers run-to-run consistency within a fixed code path under documented constraints (Intel Corporation, 2023). The PLASMA and MAGMA projects (Agullo et al., 2009) took the opposite design stance for performance: dynamic, dependency-driven scheduling that deliberately does not fix reduction order, which is excellent for throughput and fatal for bit-reproducibility. On the scalar side, correctly rounded elementary-function libraries—CR-LIBM (Daramy et al., 2003), RLIBM (Lim and Nagarakatte, 2021), and the CORE-MATH project (The CORE-MATH Project, 2022)—show that the transcendentals can be pinned down to the last bit, at a cost.

Urbit is distinguished by its requirement that determinism be not one option among many but a first-class feature of the OS and runtime. This requirement is structurally enforced by the Nock ISA-jetted runtime relationship. Jets utilize relatively faster code paths and reductions but produce the identical result as the slow path: the specification is authoritative. Thus

linear algebra techniques, even those following the BLAS pseudo-standard, must follow a reproducible fast path on all supported architectures.

3 Deterministic Architecture

A Nock computation is a pure function from noun to noun. A *jet* is a native routine registered against a named Hoon core; when the interpreter evaluates that core, it may run the jet instead of the Nock. The iron discipline that makes this sound is simple: the jet must produce the same noun as the Hoon, for every input, or it is a bug in the runtime. A divergence between jet and specification is termed a *jet mismatch* and is treated as a correctness failure, not a numerical tolerance.

The Hoon (or its compiled result as Nock) is therefore the specification, in the strong sense that it defines what the right answer *is*. The C jet is an accelerant with no semantic authority. In some cases, the jet may decline: if a native routine returns the sentinel `u3_none`, the interpreter falls through and evaluates the Hoon directly. The consequence for numerics is that an arithmetic kind with no jet, or with a jet that punts on some inputs, is not unsupported—it is merely slow. The specification still runs, and the answer is still defined and still deterministic, because Hoon over exact nouns is deterministic by construction.

This crystallizes, or perhaps even inverts, the usual relationship between correctness and performance. In conventional HPC, one has an optimized C or Fortran program, the optimized C arguably *is* the program and its source reference implementation is documentation (to say nothing of its documentation). There may of course be bugs in the compiled code, but the optimized program is the concrete artifact. By contrast in Urbit, the optimized C program must justify itself against an independent authoritative definition. Determinism thus follows from two facts: the specification is a pure function of exact data, and the jet is contractually bit-identical to the specification. A host without the jet computes the same bits more slowly; a host with a divergent jet is non-conforming and

191 must be repaired.

192 For our linked implementations like SoftBLAS, this means
193 the library is never permitted to be “approximately” the speci-
194 fication. A `gemm` jet that summed its inner products in a hardware-
195 chosen order would be a mismatch even if every individual
196 rounding were IEEE 754-correct, because the Hoon specifica-
197 tion sums them in one fixed order and the jet must reproduce
198 *that* number.⁷ Our BLAS must satisfy the constraint of deter-
199 minism to be admissible as a jet at all; thus, SoftBLAS was de-
200 veloped on top of SoftFloat (Hauser, 2018). This yields correct-
201 ness and consistency rather than hardware-optimized perfor-
202 mance.

203 3.1 The numerical stack

204 Urbit natively supports IEEE 754 floating-point mathematics
205 in its `%base` distribution, and through the supplied numerical
206 stack additionally supports unums (posits and quires); complex
207 IEEE 754; two’s complement integers; and fixed-point mathe-
208 matics.

209 The numerical libraries are layered as follows; each layer
210 is Hoon-normative with optional, independently verifiable C
211 jets.

212 1. *Libraries*. Imported from `/lib` at compile time.

213 (a) `/lib/math`. The basic IEEE 754 arithmetic and tran-
214 scendental functions library supports four preci-
215 sions (16-bit `@rh`, 32-bit `@rs`, 64-bit `@rd`, and 128-
216 bit `@rq`) across a battery of functions like exponen-
217 tiation and logarithm, trigonometric and inverse-
218 trigonometric functions, and rounding and com-
219 parison utilities. Transcendental functions are im-
220 plemented as Chebyshev minimax polynomials with
221 Cody–Waite range reduction (Cody and Waite, 1980).
222 Jets run on SoftFloat (Hauser, 2018). Originally com-
223 posed by `~lagrev-nocfep` and `~mopfel-winrux`.

⁷Summation will necessarily exhibit order-dependent drift at the bit level.

- 224 (b) `/lib/twoc`. A two's-complement integer arithmetic
 225 package supporting scalar operations. Jetted with
 226 an internal library against GMP. Originally com-
 227 posed by `~litlep-nibbyt`.
 - 228 (c) `/lib/fixed`. Fixed-point (Q-format) arithmetic at
 229 power-of-two total widths. Unjetted beyond stan-
 230 dard integer arithmetic jets because de facto integer
 231 mathematics hold. Originally composed by `~lagrev-`
 232 `nocfep`.
 - 233 (d) `/lib/unum`. 2022-standard posit arithmetic (`, 2022`)
 234 at five widths (8-bit `@rpb`; 16-bit `@rph`; 32-bit `@rps`;
 235 64-bit `@rpd`; 128-bit `@rpq`, i.e. `posit8` through `posit128`),
 236 `es = 2` throughout, with quire accumulation. See
 237 Section ?? on oracle coverage. Jetted with Soft-
 238 Unum. Originally composed by `~lagrev-nocfep`.
 - 239 (e) `/lib/complex`. Complex arithmetic over the four
 240 IEEE 754 precisions, riding on the scalar float layer.
 241 Originally composed by `~lagrev-nocfep`.
- 242 2. *Lagoon (Linear ALgebra in hOON)*. A rank- N array li-
 243 brary (a `$ray`) carrying a `$kind` field that dispatches to
 244 the appropriate arithmetic: `%i754`, `%uint`, `%int2`, `%unum`,
 245 `%cplx`, `%fixp`. Storage is row-major and identical in mem-
 246 ory layout to underlying runtime arrays; the `%i754` Level-
 247 3 path is jetted through SoftBLAS and the `%unum` path is
 248 jetted through SoftUnum.
 - 249 3. *Saloon (Scientific ALgorithms in hOON)*. Scientific algo-
 250 rithms over Lagoon rays. Types correspond to the La-
 251 goon field. Element-wise transcendentals are dispatched
 252 through `/lib/math`, and dense routines such as the Ja-
 253 cobi symmetric eigendecomposition.⁸
 - 254 4. *Maroon (MAchine leaRning in hOON)*. An inchoate ma-
 255 chine learning library built atop Lagoon and Saloon.

256 The single most consequential design choice is Lagoon's
 257 `$kind` dispatch with fall-through (Listing 1). One basic array

⁸Broadly speaking, `Lagoon::NumPy::Saloon::SciPy`.

type serves six arithmetics; adding a seventh requires only a Hoon implementation, and a jet for it is optional and separately auditable. This is architecturally distinct from NumPy (which supplies one dtype per array with no in-band kind tag); from Julia (which has compile-time specialization per concrete type); and from the APL/J/K tradition (which supply a uniform numeric tower with no explicit kind dispatch). Because an unjetted kind still runs its Hoon, the specification is authoritative for every kind, jetted or not. Two other features of \$ray's design call attention: the MSB pinned 1 bit, which preserves otherwise-stripped leading zeroes in atoms and thus makes the atom representation exactly match the expected C memory layout of an array; and the tail=* arbitrary noun specialization field, which preserves jet axis addresses in case of future extension.⁹

Listing 1: Lagoon base types. From /sur/lagoon.

```
273 +$ ray :: $ray: n-dimensional array
274 $: =meta :: descriptor
275 data=@ux :: data, row-major order
276 ==
277 ::
278 +$ prec [a=@ b=@] :: fixed-point precision
279 +$ meta :: $meta: metadata for a $ray
280 $: shape=(list @) :: list of dimension lengths
281 =bloq :: logarithm of bitwidth
282 =kind :: name of data type
283 tail=* :: per-kind specialization data
284 ==
285 ::
286 +$ kind :: $kind: type of array
287 scalars
288 $? %i754 :: IEEE 754 float
289 %uint :: unsigned integer
290 %int2 :: 2s-complement integer
291 (/lib/twoc)
292 %unum :: unum/posit (/lib/unum)
293 %cplx :: complex (/lib/complex)
```

⁹For %cplx, bloq is the $\log_2$ of the packed-pair element width (both components combined). Thus @cs (two @rs components) has bloq=6, while each component @rs has bloq=5.

```

294         %fixp                :: fixed-point Q a.b
295         (/lib/fixed)
296     ==
297     ::
298     +$ baum                    :: $baum: ndarray with metadata
299     $: =meta                  ::
300     data=ndray                ::
301     ==
302     ::
303     +$ ndarray                :: $ndray: n-dim array as list
304     $@ @                      :: single item
305     (list ndarray)            :: nonempty list of children

```

3.2 SoftBLAS and SoftUnum

At the current time, two external numerical libraries facilitate

3.2.1 SoftBLAS

SoftBLAS is built on Berkeley SoftFloat 3e (Hauser, 2018). Following SoftFloat, it requires a C99-compliant host and passes all floating-point quantities by value. Every arithmetic primitive used by every routine is a SoftFloat call: there is no path to a hardware floating-point instruction, and therefore no path to hardware-dependent rounding, contraction, or intermediate precision. For reasons enumerated by [~lagrev-nocfer \(2024\)](#), this is the only current method which guarantees that the output of a floating-point computation is a pure function of its inputs, and therefore deterministic across hosts.

SoftBLAS covers the four IEEE 754 binary precisions used across the stack: half (h), single (s), double (d), and quadruple (q).¹¹ Routines follow the BLAS prefixing convention extended to the half and quad types. Real-valued routines across all three levels are complete as of the cited release; complex-valued rou-

¹⁰Urbit mathematics are supported internally by the GNU Multiple Precision Arithmetic Library and SoftFloat.

¹¹Complex routines are supplied at half (i), single (c), double (z), and quadruple (v) precisions.

times are in progress but sufficient to support Lagoon’s requirements.

Table 1: SoftBLAS coverage by level (real-valued).

Level	Cost	Representative routines
1	$O(n)$	?dot, ?axpy, ?scal, ?nrm2, ?asum, etc.
2	$O(n^2)$	?gemv
3	$O(n^3)$	?gemm

The rounding mode is a global, `softblas_roundingMode` (a `char` typedef declared in `softblas.h`) interpreted by the underlying SoftFloat layer.¹² The admissible values are the SoftFloat 3e rounding modes (round to nearest even, round toward zero, round toward $+\infty$, round toward $-\infty$, and round to nearest-away).¹³ The default is round to nearest-even, %n. The Hoon specification names the rounding mode in the %base distribution, and the jet is required to honor it.

SoftBLAS satisfies three conditions that conventional BLAS libraries do not:

1. *Fixed reduction order.* Every reduction (dot products, gemm inner loops, norms) sums in one canonical order, identical to the order the Hoon specification uses. No tiling or threading is permitted to reassociate it. (Urbit runtimes are, in any case, single-threaded at the time of writing.)
2. *No contraction.* Multiply-add is two roundings, never a fused one, because the specification rounds twice. There is no FMA path, which as ~lagrev-nocfep (2024) exhibited is prone to nondeterminism.
3. *Software rounding.* All rounding is performed by SoftFloat in software, so it is identical on every host regardless of native floating-point behavior.

¹²The transcendental function implementations ignore the rounding mode, which thus applies only to the arithmetic operations.

¹³IEEE 754-2008 introduced another mode, round to nearest ties to max magnitude, which is not included in Urbit’s toolset.

3.2.2 SoftUnum

SoftUnum is a software-defined universal number (unum) library that implements the 2022 posit standard (, 2022). It supports five widths (8, 16, 32, 64, and 128 bits) with $es = 2$ throughout. The quire accumulator is implemented in software and is bit-identical across hosts. The library is jetted into Lagoon's %unum kind.

4 Types

The mathematical base types are enumerated in Listing 2. These have occasioned the introduction of several new atom auras, given in Table 2. We have not introduced an aura for two's-complement integers at the current time, but these will likely nest under @s (signed integer) in the future. The @rqb through @rqq auras are quire accumulators for the corresponding posit widths, and are intended for internal algorithms not general use.¹⁴

Listing 2: Mathematical base types. From /sur/lagoon.

```

365 +$ kind          :: $kind: type of array
366     scalars
367     $? %i754      :: IEEE 754 float
368     %uint         :: unsigned integer
369     %int2         :: 2s-complement integer
370     (/lib/twoc)
371     %unum         :: unum/posit (/lib/unum)
372     %cplx         :: complex (/lib/complex)
373     %fixp         :: fixed-point Q a.b
374     (/lib/fixed)
375     ==
    
```

Aura types have been named after the target width in the cases of complex numbers and quires.

¹⁴A quire is a $16n$ -bit two's-complement fixed-point accumulator with scale $2^{-qscale}$ ($qscale = 8n - 16$): a posit value v contributes the exact integer $v * 2^q scale$.

Table 2: Existing and new atom auras. From `/sur/lagoon`.

Atom aura	Description
<code>@rh</code>	16-bit IEEE 754 float
<code>@rs</code>	32-bit IEEE 754 float
<code>@rd</code>	64-bit IEEE 754 float
<code>@rq</code>	128-bit IEEE 754 float
<code>@rpb</code>	8-bit posit
<code>@rph</code>	16-bit posit
<code>@rps</code>	32-bit posit
<code>@rpd</code>	64-bit posit
<code>@rpq</code>	128-bit posit
<code>@rqbH</code>	8-bit quire (128-bit accumulator for <code>@rpb</code>)
<code>@rqhI</code>	16-bit quire (256-bit accumulator for <code>@rph</code>)
<code>@rqsJ</code>	32-bit quire (512-bit accumulator for <code>@rps</code>)
<code>@rqdK</code>	64-bit quire (1024-bit accumulator for <code>@rpd</code>)
<code>@rqqlL</code>	128-bit quire (2048-bit accumulator for <code>@rpq</code>)
<code>@ch</code>	32-bit complex (two 16-bit floats)
<code>@cs</code>	64-bit complex (two 32-bit floats)
<code>@cd</code>	128-bit complex (two 64-bit floats)
<code>@cq</code>	256-bit complex (two 128-bit floats)

4.1 Verification

The determinism architecture reduces correctness to one question per routine: does the jet equal the specification on every input? We answer it insofar as possible with oracles external to both the Hoon and C implementations. SoftFloat by and large served as the oracle for the IEEE 754 floating-point operations (real and complex), while the posit mathematics were verified against two independent references: SoftPosit (Leong, 2018) (`posit8`, `posit16`, `posit32`) and the Universal library (Omtzigt et al., 2020) (`posit64`, `posit128`).

The quire’s defining property—exact accumulation until a single final rounding—is checked directly against an exact rational sum.

Elementary functions are checked against MPFR (Fousse et

al., 2007) at high precision. At half precision the input space is 2^{16} values and the test is exhaustive; at single precision the 2^{32} space is likewise tested in full. At double and quadruple precision, where exhaustive testing is infeasible, we report maximum observed unit in the last place (ULP) error over a sampled and edge-weighted input set rather than a guarantee. We claim a faithful-rounding (≤ 1 ULP) result where the testing supports it and a measured bound where it does not.

5 Contributions

This deterministic numerical stack contributes several properties to the broader discussion of numerical determinism and reproducibility. Determinism is a structural property of Urbit: while reproducible floating point is a standing goal of several parties (Demmel and Nguyen (2013); Intel Corporation (2023)), Urbit contributes a philosophical first-class enforcement mechanism.¹⁵

Although not novel, the library covers a broad range of bit widths and precisions, including half-precision and quadruple-precision, which are not widely supported in other numerical computing libraries. Likewise we provided faithfully-rounded elementary functions at all precisions.

Lagoon’s heterogeneous-kind array design with a single `$ray` type is a novel approach to array libraries, allowing for in-band kind dispatch and fall-through to specification. As mentioned previously, this permits more flexibility and extensibility in the base representation than NumPy, Julia, or the APL/J/K tradition. The `kind` field allows for easy addition of new numeric types without requiring changes to the underlying array structure, and the fall-through mechanism ensures that un-jetted kinds can still be computed correctly using the Hoon specification.

¹⁵Jet co-design methodology is treated separately; see `~lagrev-nocfep`, pp. TODO-TODO in this volume, for a companion article.

6 Benchmarks

While a software-defined floating-point library will never match the performance of a hardware-accelerated one, we have nevertheless seen impressive speedups by refining the algorithms and the runtime jets. The benchmarks reported here are for a single-threaded, single-core execution on a modern 64-bit compiler CPU (AArch64, Apple M4 Max). They should serve as order-of-magnitude indications on contemporary hardware.

The first benchmark results tabulate what the jetted system buys over interpreting the Hoon specification directly. Table 3 reports jetted versus interpreted time for `IEEE 754 /lib/math`. For most trials, 100,000 runs were evaluated and averaged. For the very slowest functions (e.g. half-precision Taylor series inverse sine), this number was modulated downwards substantially.¹⁶ The original `/lib/math` and Saloon implementations of transcendentals were naïve Taylor series expansions and serve as a worst-case demonstration. Accuracies are comparable for the number of Taylor series terms utilized to achieve an appropriate relative tolerance, but the Chebyshev basis functions are uniformly faster and more correct. All values reported are measured within a running Urbit ship (rather than from an independent test harness).

¹⁶Exhaustive results for all precisions are available at `urbit/numerics@main:benchmark`.

Table 3: Elapsed times and relative slowdowns of IEEE 754 jetted Chebyshev functions for double-precision @rd atom operations with /lib/math, interpreted Chebyshev functions, and interpreted Taylor series functions. Slowdown is reported relative to the fastest case, the jetted Chebyshev functions.

Arm	Chebyshev (jet)	Chebyshev (int.)	Taylor (int.)	C. slowdown	T. slowdown
+exp	1.67 μ s	296 μ s	11.0 ms	177 ^x	6,565 ^x
+log	1.76 μ s	164 μ s	13.3 ms	94 ^x	7,600 ^x
+sin	1.95 μ s	278 μ s	0.94 ms	143 ^x	480 ^x
+cos	1.97 μ s	275 μ s	0.94 ms	140 ^x	476 ^x
+tan	4.53 μ s	241 μ s	1.30 ms	53 ^x	288 ^x
+atan	2.23 μ s	47.6 μ s	109 ms	21 ^x	48,784 ^x
+atan2	2.72 μ s	56.2 μ s	11.5 ms	21 ^x	4,237 ^x
+asin	2.07 μ s	54.8 μ s	205 ms	27 ^x	99,130 ^x
+acos	2.10 μ s	51.2 μ s	5.17 ms	24 ^x	2,459 ^x
+sqrt	1.92 μ s	10.3 μ s	73.5 μ s	5 ^x	38 ^x
+cbrt	2.44 μ s	467 μ s	111 ms	192 ^x	45,723 ^x
+pow	2.87 μ s	656 μ s	1.68 ms	229 ^x	585 ^x
+pow-n	2.62 μ s	217 μ s	219 μ s	83 ^x	83 ^x
+log-2	2.34 μ s	164 μ s	13.3 ms	70 ^x	5,668 ^x
+log-10	2.38 μ s	165 μ s	13.4 ms	69 ^x	5,638 ^x

445 The `/lib/unum` transcendentals are currently implemented
446 as Chebyshev basis expansions like the `/lib/math` operations.
447 Results are included in Table 4.

Table 4: Elapsed times and relative slowdowns of unum jetted Chebyshev functions for 32-bit @rps posit32 operations with /lib/unum, interpreted Chebyshev functions, jetted Taylor series functions, and interpreted Taylor series functions. Slowdown is reported relative to the jetted Chebyshev functions.

Arm	Chebyshev (jet)	Chebyshev (int.)	Taylor (jet)	Taylor (int.)	C. slowdown	T. slowdown
+exp	16.5 μ s	550 μ s	25.3 μ s	19.4 ms	33 $\times$	1,176 $\times$
+log	17.9 μ s	739 μ s	39.9 μ s	39.1 ms	41 $\times$	2,184 $\times$
+sin	18.3 μ s	813 μ s	34.8 μ s	32.3 ms	44 $\times$	1,765 $\times$
+cos	18.3 μ s	817 μ s	35.7 μ s	32.5 ms	45 $\times$	1,775 $\times$
+atan	18.3 μ s	832 μ s	850.3 μ s	63.7 ms	45 $\times$	3,480 $\times$
+asin	39.6 μ s	2.01 ms	—	—	50 $\times$	—
+acos	40.9 μ s	2.15 ms	—	—	52 $\times$	—
+sqrt	17.1 μ s	209 μ s	17.3 μ s	341 μ s	12 $\times$	20 $\times$
+cbt	32.0 μ s	1.76 ms	—	—	55 $\times$	—
+pow	31.2 μ s	1.40 ms	60.7 μ s	56.9 ms	45 $\times$	1,824 $\times$
+pow-n	2.81 μ s	698 μ s	—	—	248 $\times$	—
+log-2	18.1 μ s	743 μ s	—	—	41 $\times$	—
+log-10	18.1 μ s	756 μ s	—	—	42 $\times$	—
+fdp	259 ns	157 μ s	270 ns	205 μ s	606 $\times$	—

6.1 Transcendental accuracy

TODO

The libraries instrumented with the Chebyshev basis produce “faithfully rounded functions” rather than “correctly rounded functions”. The distinction is that a faithfully rounded result is within 1 ULP of the exact result (Rump, Ogita, and Oishi, 2008), while a correctly rounded result is the exact representable value closest to the exact result. Faithful rounding is often sufficient for numerical applications and is generally faster to compute than correct rounding. Truly correct rounding is often prohibitively expensive, especially for transcendental functions, and can require much higher intermediate precision to attain.¹⁷

6.2 The price of determinism

The third field of benchmarking measures the overhead of the determinism guarantee relative to a conventional, nondeterministic, hardware BLAS. We report SoftBLAS against a stock optimized BLAS on the same host. We expect SoftBLAS to be slower; here we quantify *how much* (our implementation of) determinism costs.

TODO

7 Future Work

We have focused more than our taste on IEEE 754 given its ongoing dominance in numerical computation, but we anticipate the libraries beyond `/lib/math` to grow in relative importance in the future.

The complex-valued SoftBLAS routines are in progress and will complete the Level 1–3 coverage. Quire operations in `/lib/unum` are implemented within the generic posit core but are not yet surfaced as callable top-level arms; exposing them is a prerequisite both for user access and for routing them

¹⁷See CR-LIBM (Daramy et al., 2003), RLIBM (Lim and Nagarakatte, 2021), and the CORE-MATH project (The CORE-MATH Project, 2022) for further details.

through the verification harness. The unum specification also defines valids, which are arithmetic intervals which have not yet seen widespread adoption.

Saloon will grow in methods adjacent to scientific computing: conjugate gradient minimization and other optimization algorithms; random number generators; and the like. The fixed-point library is relatively underdeveloped compared to the other arithmetics, and will be expanded to support more general Q-format widths and operations. More components of the various libraries need to be upgraded to a state-of-the-art Chebyshev basis and benchmarked for performance.

Finally, Maroon, the machine learning library, is in its infancy and will be developed to support a range of machine learning algorithms and techniques. Part of this field of consideration encompasses the utilization of GPU programming for LLM-based computing within or adjacent to Urbit. GPU numerical nondeterminism and host portability presents a challenge of the same order of magnitude as the baseline reproducibility we have tackled over the past several years in this project. One can envision a ReproCL as a deterministic counterpart to OpenCL, but it is unclear whether sufficient warrant exists to justify the effort as of the mid-2020s. Heterogeneous compute would be benefitted by the adoption of alternatives to IEEE 754 with better guarantees of portability: not just unums but `bf16` and other newer types may answer the need.

8 Conclusion

Urbit as a deterministic state machine cannot treat numerical determinism or bitwise reproducibility as merely a tolerance. The architectural stack we have described here constitutes a partial answer to the challenge underlying `~lagrev-nocfep` (2024): how can a numerical computation built on a particular stack be reliably reproducible? We maintain this determinism by painstaking construction. The computational cost of determinism is currently (and regrettably) non-negligible, while one can hope for future developments, protocols, and architectures that make it more natural and tenable. ☞

References

- Agullo, Emmanuel et al. (2009). “Numerical Linear Algebra on Emerging Architectures: The PLASMA and MAGMA Projects.” In: *Journal of Physics: Conference Series* 180, p. 012037. DOI: 10.1088/1742-6596/180/1/012037.
- Charney, J., R. Fjortoft, and J. von Neumann (1950). “Numerical Integration of the Barotropic Vorticity Equation.” In: *Tellus* 2.4, pp. 237–254. DOI: 10.1111/j.2153-3490.1950.tb00336.x. URL: <https://doi.org/10.1111/j.2153-3490.1950.tb00336.x>.
- Cody, William J. and William M. Waite (1980). *Software Manual for the Elementary Functions*. Prentice-Hall.
- Daramy, Catherine et al. (2003). “CR-LIBM: A Correctly Rounded Elementary Function Library.” In: *Proc. SPIE 5205, Advanced Signal Processing Algorithms, Architectures, and Implementations XIII*. DOI: 10.1117/12.505591.
- Demmel, James and Hong Diep Nguyen (2013). “Fast Reproducible Floating-Point Summation.” In: *2013 IEEE 21st Symposium on Computer Arithmetic (ARITH)*. ReproBLAS project, pp. 163–172. DOI: 10.1109/ARITH.2013.9.
- Fousse, Laurent et al. (2007). “MPFR: A Multiple-Precision Binary Floating-Point Library with Correct Rounding.” In: *ACM Transactions on Mathematical Software* 33.2, 13:1–13:15. DOI: 10.1145/1236463.1236468.
- Hauser, John R. (2018). *Berkeley SoftFloat, Release 3e*. <http://www.jhauser.us/arithmetic/SoftFloat.html>.
- IEEE Standard for Floating-Point Arithmetic (2019). *IEEE Std 754-2019*. IEEE. DOI: 10.1109/IEEESTD.2019.8766229.
- Intel Corporation (2023). *Conditional Numerical Reproducibility (CNR) in Intel oneAPI Math Kernel Library*. Intel Developer Documentation.
- ~lagrev-nocfep, N. E. Davis (2024). “The Desert of the Reals: Floating-Point Arithmetic on Deterministic Systems.” In: *Urbis Systems Technical Journal* 1.1, pp. 93–131.
- Lawson, C. L. et al. (1979). “Basic Linear Algebra Subprograms for Fortran Usage.” In: *ACM Transactions on Mathematical Software* 5.3, pp. 308–323. DOI: 10.1145/355841.355847.

- Leong, S. H. (Cerlane) (2018). *SoftPosit: A Software Implementation of Posit Arithmetic*.
<https://gitlab.com/cerlane/SoftPosit>.
- Lim, Jay P. and Santosh Nagarakatte (2021). “High Performance Correctly Rounded Math Libraries for 32-bit Floating Point Representations.” In: *Proc. 42nd ACM SIGPLAN Conf. on Programming Language Design and Implementation (PLDI)*, pp. 359–374. DOI: 10.1145/3453483.3454049.
- Omtzigt, E. Theodore L. et al. (2020). “Universal Numbers Library: Design and Implementation of a High-Performance Reproducible Number Systems Library.” In: *arXiv preprint arXiv:2012.11011*. URL: <https://arxiv.org/abs/2012.11011>.
- Rump, Siegfried M., Takeshi Ogita, and Shin’ichi Oishi (2008). “Accurate Floating-Point Summation Part I: Faithful Rounding.” In: *SIAM Journal on Scientific Computing* 31.1, pp. 189–224. DOI: 10.1137/050645671. URL: <https://doi.org/10.1137/050645671>.
- ~sorreg-namtyv, Curtis Yarvin et al. (2016). *Urbit: A Solid-State Interpreter*. Whitepaper. Tlon Corporation. URL: <https://media.urbit.org/whitepaper.pdf> (visited on ~2024.1.25).
- Standard for Posit Arithmetic (2022)* (2022). Posit Working Group. URL: https://posithub.org/docs/posit_standard-2.pdf.
- The CORE-MATH Project (2022). *CORE-MATH: Correctly Rounded Mathematical Functions*.
<https://core-math.gitlabpages.inria.fr/>.